Projet L3 : Cestionnaire C++

COURBET Nathan
WILBAUT Cristophe

Sommaire

Projet L3 : Cestionnaire C++....1
1. Description et Objectif....3
2. Pémisse du projet....3
3. Cahier des charges....4
3.1 Fonctionnalitées....4
3.2 Problèmatiques....4
4. Développement....5
4.1 Outils envisagés (peut changer au cours du temps)....5
4.2 Base de l'application....5
4.3 Architecture de l'application....6
4.4 Création du générateur de mot de passe....7
4.5 Sauvgarde, modification, suppression....8
4.6 Hachage et création d'utilisateur....9
4.7 Interface Graphique....13
4.8 Difficultées rencontrées....15
5. Perspectives & améliorations....16
6. Conclusion....17
7. Annexes....17
7.1 Lien Github....17
7.2 Sources :....17

1. Description et Objectif

Dans le cadre de ma troisième année de Licence Informatique, je réalise un projet intitulé « Cestionnaire » (jeux de mot avec gestionnaire et le langage C), c'est une application de gestion de mots de passe destinée à faciliter et sécuriser la gestion des identifiants personnels et professionnels des utilisateurs.

L'objectif principal de cette application est de créer, organiser, stocker et modifier les différents mots de passe d'un utilisateur à travers une interface graphique intuitive et ergonomique.

Le projet mettra l'accent sur la sécurité des données sensibles, en proposant deux modes de stockage :

- Localement, sur l'appareil de l'utilisateur
- À distance, sur une base de données sécurisée, selon les préférences et besoins de l'utilisateur.

L'application intégrera plusieurs mécanismes de protection tel que des algorithmes de chiffrements pour citer un exemple, qui seront mises en place afin de garantir la confidentialité et l'intégrité des informations.

À terme, "Cestionnaire" devrait se présenter comme une possibilitée sécurisée et conviviale de gestion des mots de passe, adaptée aussi bien aux particuliers qu'aux professionnels.

2. Pémisse du projet

Les raisons qui m'ont poussé à m'impliquer dans ce projet sont tout d'abord un besoin, la plupars des gestionnaires de mot de passe ne me donnaient pas envie ou bien étaient payant, j'ai donc eu l'idée de crée le mien.

Cela me procure deux avantages :

- Le savoir total sur comment les mots de passes sont géré dans mon application
- En apprendre beaucoup plus sur la cybersécurité en particulier la sécurité des données utilisateur

Ces différentes raisons m'ont donc poussé à crée mon propre gestionnaire de mot de passe.

3. Cahier des charges

3.1 Fonctionnalitées

- Génération de mots de passe sécurisé.
- Ajout, suppression, modification des entrées et mot de passe.
- Catégorisation des mots de passe (Tri par sites)
- Recherche et filtrage des informations.
- Le chiffrement des mots de passe et leur stockage

3.2 Problèmatiques

- Protection des mots de passes
- Gestion des sessions utilisateurs
- Respecter la RGPD
- Crée un interface ergonomique

4. Développement

4.1 Outils envisagés (peut changer au cours du temps)

Pour le dévloppement du « Cestionnaire » j'ai choisis différents outils qui me permettent une création plus organisée et efficace de l'application.

Nous pouvons noter en particulier,

- **Kate**, un IDE simpliste qui me permet de faire du c++ de manière agrèable.
- **Github** et **Git** pour avoir une gestion de versions et fichers.
- **Apache** un outil de creation de serveur.
- **Qt** un outil de création d'interface graphique.
- **MariaDB** une base de donnée pour le serveur.

4.2 Base de l'application

Pour simplifier le développement de l'application, nous procédons par étapes. Cela ne sert à rien, et serait même contre-productif, de tout faire d'un coup.

Ma méthode de travail consiste donc à créer les fonctionnalités de base : la création, la modification et le stockage des différents mots de passe, sans se soucier de l'UI/UX (interface utilisateur).

L'application doit déjà être fonctionnelle dans le terminal avant d'ajouter une UI/UX. Cela permet de maintenir le code « propre », c'est-à-dire sans le code pour l'interface graphique, ce qui compliquerait énormément le développement, car le code serait beaucoup plus complexe.

Au lieu de ça, nous nous concentrons uniquement sur les fonctionnalités les plus importantes. L'UI/UX n'est nécessaire que pour simplifier l'utilisation.

4.3 Architecture de l'application

Pour l'architecture, j'ai diviser les reccourses en différents sous dossiers avec chacuns leur rôle dans le fonctionnement de l'application.

Sur l'image ci contre vous pouvez visualiser la structure de l'application.

```
.
├── build
│   ├── app.o
│   ├── cestionaire
│   ├── cestionaire.pro
│   ├── crypto.o
│   ├── jsonGestioner.o
│   ├── main
│   ├── main.o
│   ├── mainwindow.o
│   ├── Makefile
│   ├── moc_mainwindow.cpp
│   ├── moc_mainwindow.o
│   ├── moc_predefs.h
│   ├── passgen.o
│   └── src
│       ├── app.o
│       ├── crypto.o
│       ├── jsonGestioner.o
│       └── passgen.o
├── json
│   ├── json.hpp
│   └── mdp.json
├── main.cpp
├── Makefile
├── rapport
│   ├── INSA_logo.png
│   ├── RapportCestionaire_Courbet-Nathan.odt
│   ├── RapportCestionaire_Courbet-Nathan.pdf
│   └── UPHF_logo.png
├── README.md
├── src
│   ├── app.cpp
│   ├── App.hpp
│   ├── crypto.cpp
│   ├── Crypto.hpp
│   ├── jsonGestioner.cpp
│   ├── JsonGestioner.hpp
│   ├── passgen.cpp
│   ├── Passgen.hpp
│   └── qtWindow
│       ├── mainwindow.cpp
│       └── mainwindow.h
└── users
    ├── benjamin
    │   └── mdp.json
    └── testUser
        └── mdp.json

10 directories, 38 files
```

Elle se compose en différents dossier tel que :

build : Contient les fichiers compliées ansi que le buil executable de l'application

Json : Librairie pour gérer les fichiers json en C

rapport : Contient le rapport du projet

src (source) : Contient le code source

users : Tout les user de l'application en local

La structure est amennée a changer au cours du développement

4.4 Création du générateur de mot de passe

Nous commençons, à mon sens, par la partie la plus simple de ce projet : le générateur de mots de passe aléatoires.

Comme son nom l'indique, nous voulons que l'application crée un mot de passe en fonction des besoins de l'utilisateur.

```
void passGen(int *len, int *maj, int *num, int *spe){
    std::string mdp;

    std::string let = "abcdefghijklmnopqrstuvwxyz";
    std::string majLet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    std::string numb = "0123456789";
    std::string spec = "!@#$%^&*()-_=+[]{};:,.<>?/";

    if (*maj) let += majLet;
    if (*num) let += numb;
    if (*spe) let += spec;

    for(int i=0; i<*len; i++){
        mdp += let[rand() % let.size()];
    }
    std::cout << mdp << std::endl;
}
```

Sans trop rentrer dans les détails, cette fonction demande à l'utilisateur la longueur du mot de passe souhaitée et s'il veut ajouter des chiffres, des lettres majuscules ou des caractères spéciaux.

Ensuite, elle « pioche » aléatoirement dans les chaînes de caractères correspondantes et affiche le mot de passe généré dans le terminal.

Nous avons donc par exemple un résultat comme celui ci :

```
longeur du mdp : 8
voulez vous ajouter des majuscules ? (Y, n)Y
voulez vous ajouter des nombres ? (Y, n)Y
voulez vous ajouter des speciaux ? (Y, n)n


 votre mdp: g9myhJD7
```

Ici, le mot de passe contient bien les options demandées par l'utilisateur, c'est-à-dire des majuscules et des chiffres.

Le mot de passe obtenu est : **`g9myhJD7`**

4.5 Sauvgarde, modification, suppression

En ce qui concerne la partie principale du projet, c'est-à-dire la gestion de mots de passe, j'ai opté dans un premier temps pour le stockage local sous forme de JSON. Pour m'aider, j'utilise la bibliothèque JSON nommée « Json.h ».

À terme, je souhaiterais chiffrer le fichier JSON avec le mot de passe maître (Master) ; c'est-à-dire un mot de passe principal du compte. Pour l'utilisateur, il n'y aura donc qu'un seul mot de passe à retenir pour accéder à toutes les données.

Avant de faire ça, nous devons déjà pouvoir interagir avec le fichier JSON qui contient les mots de passe : la première étape consiste donc à sauvegarder une entrée.

La fonction se présente sous cette forme :

```
void ajouterEntree(const std::string& cheminFichier, std::string& site, std::string& username, std::string& password, std::string& note){
    std::ifstream input(cheminFichier);
    json data;

    if(input){
        input >> data;
        input.close();
    } else {
        data["entries"] = json::array();
    }

    json entree;
    entree["site"] = site;
    entree["username"] = username;
    entree["password"] = password;
    entree["note"] = note;

    data["entries"].push_back(entree);

    std::ofstream output(cheminFichier);
    output << data.dump(4);
    output.close();

    std::cout << "Mdp bien defini !" << std::endl;

}
```

Ici, nous récupérons différents champs : le mot de passe, évidemment, mais aussi le site web sur lequel il est utilisé, le nom d'utilisateur et une note.

```
entrez le nom du site : entuphf.fr
entrez votre username : nathan.courbet
entrez votre mdp : CeciEstUnBonMotDePasse2005!
entrez une note : ent
Mdp bien defini !
+----------------------------------------+
```

Nous créeons donc une nouvelle entrée pour tester si tout se déroule bien.

```
{
    "entries": [
        {
            "note": "",
            "password": "Test1234",
            "site": "google.com",
            "username": "usergoogle"
        },
        {
            "note": "un",
            "password": "password",
            "site": "huh.com",
            "username": "huh"
        },
        {
            "note": "ent",
            "password": "CeciEstUnBonMotDePasse2005!",
            "site": "entuphf.fr",
            "username": "nathan.courbet"
        }
    ]
}
```


Nous pouvons remarquer que l’entrée s’est bel et bien éffectuée dans le fichier Json de l’utilisateur.

Nous pouvons maintenant effectuer des modifications, ajouts ou effacer les données que nous souhaitons.

4.6 Hachage et création d’utilisateur

Nous avons malheureusement un défaut important : le mot de passe de l’utilisateur est en clair dans le fichier JSON. Cela pose un gros problème de sécurité au sein de l’application, qui est pourtant le but principal de celle-ci.

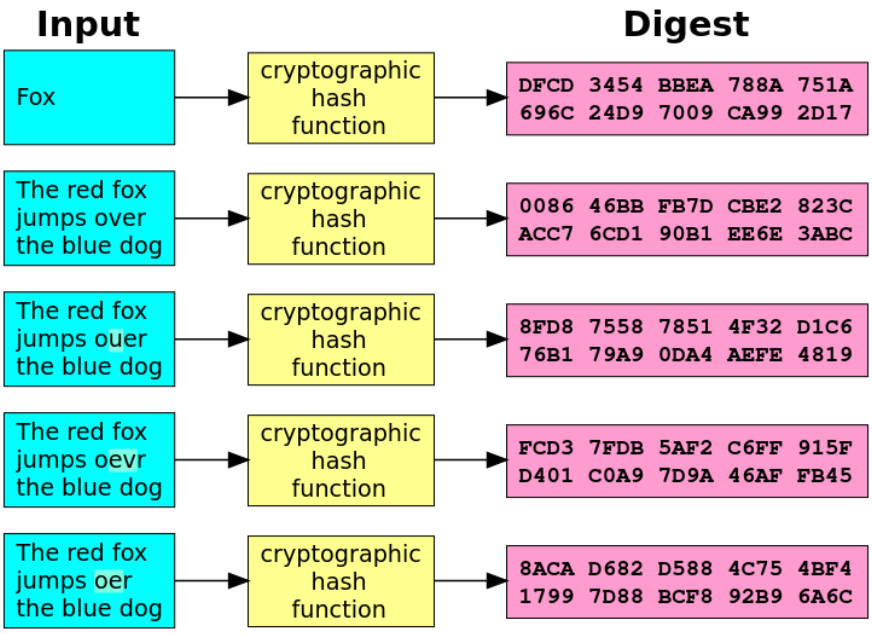
En effet, si une personne malveillante arrivait à mettre la main sur le fichier JSON, elle aurait accès à tous les mots de passe de l’utilisateur.

Il faut donc renforcer la sécurité, cacher les mots de passe pour les personnes qui ne sont pas censées avoir accès à ces données.

Le hachage est une manière de rendre une chaîne de caractères (string) invisible aux yeux d'un lecteur.

À ne pas confondre avec le chiffrement, le hachage est un algorithme qui permet de transformer une chaîne de caractères en une sortie toujours de la même taille, presque impossible à inverser pour retrouver la chaîne d'entrée.

Dans le monde, le hachage est extrêmement utilisé pour cacher des mots de passe dans une base de données et dans d'autres domaines pour protéger des données sensibles.



Source image: wikipedia

Pour le hachage et le chiffrement, j'utilise la librairie **OpenSSL**.

L'utilisateur entre en premier son mot de passe en clair.

La fonction de hachage lui applique un « sel » et le hache pour le rendre totalement illisible.

Par exemple, pour le hachage du mot de passe (« master ») :

TestUser, notre utilisateur préféré, veut se créer un compte sur l'application.

Il entre donc son pseudo « testUser » et saisit son mot de passe.

Son mot de passe est donc transformé de manière à ce qu'il ne soit pas visible pour un autre utilisateur ou quelqu'un de malveillant.

```
... users > testUser > .env
1  PASSWORD_HASH=7191865813214466004
2  SALT=2dbd7e6a58eff076abb7c237f64a716f
3
```

Maintenant, on efface son mot de passe de la mémoire et il est complètement masqué et presque irrécupérable.

```
std::fill(mainPassword.begin(), mainPassword.end(), '\0');
```

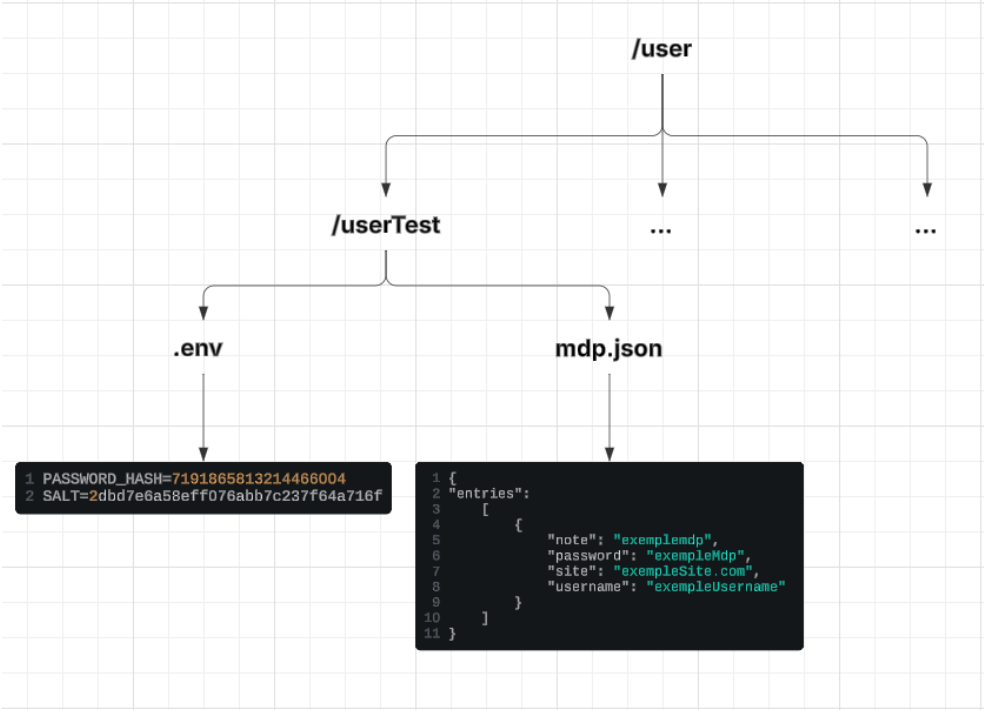
Si notre utilisateur préféré TestUser souhaite se connecter, rien de plus simple : il entre son mot de passe (« master ») et on le repasse dans la fonction de hachage.

Si le hash correspond, l'utilisateur a bien entré son bon mot de passe et la connexion est presque instantanée. Dans le cas contraire, connexion impossible.

Au niveau de la structure de stockage, chaque utilisateur aura son propre dossier géré par l'application, qui contient un fichier .env, qui est l'endroit où le mot de passe maître hashé est stocké ainsi que le sel appliqué à ce hash.

Chaque dossier utilisateur contiendra aussi un fichier mdp.json qui contient tous les mots de passe de l'utilisateur, qui sont chiffrés à partir du mot de passe maître de l'utilisateur.

La structure se représente sous cette forme :



« … » Représente les différents utilisateurs.

« /nom » Représente les dossiers.

« nom » Représente les fichiers.

Je me suis rendu compte avec le temps qu’une base de données aurait été beaucoup plus simple à gérer et à optimiser. Cela pourrait faire l’objet d’une future amélioration du projet.

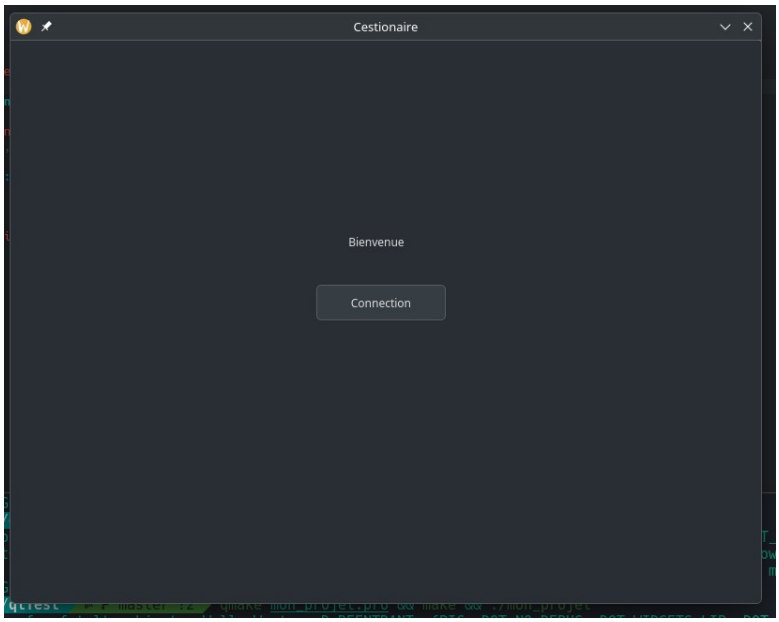
4.7 Interface Graphique

Pour ce qui est de l'interface graphique, je me suis penché sur la librairie Qt.

C'est un outil qui permet de créer des interfaces graphiques simplement en incluant des éléments qui permettent de faire une fenêtre ergonomique et fonctionnelle.

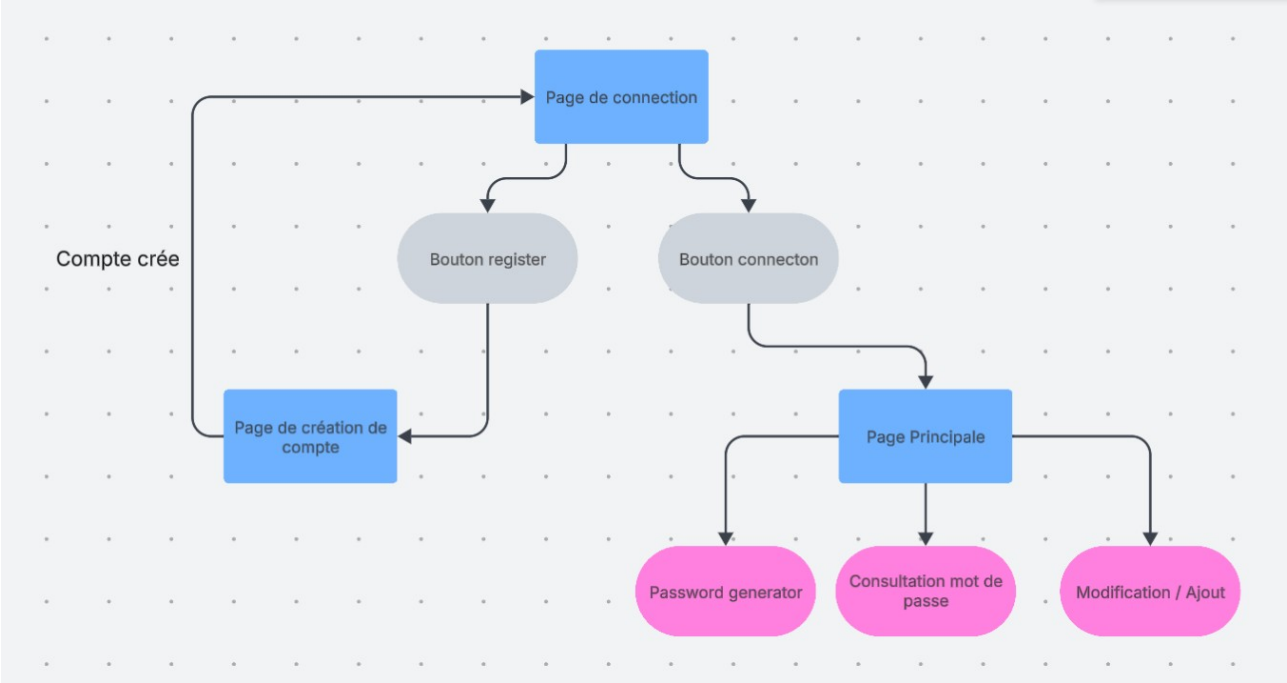
Un avantage de Qt est que les environnements créés avec cet outil sont aussi bien pris en charge par GNU/Linux que Windows ou encore macOS, ce qui répond à la contrainte de multiplateformité de l'application.

Pour utiliser Qt, nous créons donc des éléments sur une fenêtre, comme par exemple un bouton.



Nous pouvons joindre à ce bouton une fonction, nous joignons donc le boutons aux fonction déjà existantes. Et nous aurons rapidement une interface graphique fonctionelle.

Je souhaite créer mon application avec différentes pages, de manière à ce que la structure ressemble à terme à ceci :



La page de connexion est la page sur laquelle on arrive quand on lance l'application. Elle permet de faire deux choses : soit rediriger vers la création d'un compte utilisateur, soit se connecter à un compte existant.

Après la connexion, nous sommes redirigés vers la page principale, qui contient plusieurs onglets, comme un « password generator » ou la consultation de ses mots de passe.

Je me suis aidé de l'IA Claude pour améliorer et accélérer la création de l'interface graphique, qui peut parfois être fastidieuse et répétitive.

Je me suis concentré sur le plan fonctionnel de l'application.

4.8 Difficultées rencontrées

Lors de la création de l'application, je me suis retrouvé confronté à différentes difficultés.

Libraires :

L'une des difficultés rencontrées a été que je ne connaissais absolument pas la librairie Qt. J'ai donc dû tout apprendre depuis zéro, ce qui m'a pris beaucoup de temps.

Sécurité :

Cacher les mots de passe m'a posé pas mal de problèmes au niveau de la structure du code. Comment éviter les fuites de mots de passe ? Un chiffrement mal implémenté ? Le mot de passe maître encore présent en mémoire ?

De nombreuses questions se posent encore à ce stade du développement, car les mots de passe peuvent être exposés à certaines attaques qu'il m'est difficile d'évaluer avec mon niveau actuel de connaissances.

Temps :

Manque de temps, bien que nous ayons disposé d'une grande partie de l'année scolaire pour réaliser nos projets, je pense avoir sous-estimé l'ampleur du projet par rapport à mon niveau de connaissances.

Pour qu'il soit correctement finalisé et sécurisé, j'aurais dû consacrer beaucoup plus de temps au développement.

5. Perspectives & améliorations

L'application possède encore un grand potentiel d'amélioration pour le futur. On peut en identifier plusieurs axes :

Application uniquement locale :

L'application est actuellement disponible uniquement en local. Cela présente certains avantages, mais aussi des inconvénients, comme l'impossibilité d'accéder à ses mots de passe depuis une autre machine. À l'avenir, on pourrait imaginer que l'utilisateur puisse stocker ses mots de passe sur un serveur sécurisé et les récupérer depuis n'importe quel appareil, à condition de posséder ses identifiants de compte.

Changement de stockage :

Bien que le stockage fonctionne dans sa forme actuelle, il existe des solutions plus optimisées et robustes que celles mises en place durant le projet, offrant de meilleures performances et une sécurité accrue.

Amélioration de l'UI/UX :

L'interface graphique et l'expérience utilisateur peuvent être améliorées pour rendre l'application plus ergonomique et agréable à utiliser.

Ajout de fonctionnalités :

Il serait intéressant d'ajouter des fonctionnalités supplémentaires, comme la gestion avancée des données ou le chiffrement de fichiers et dossiers, afin d'enrichir l'utilité de l'application.

6. Conclusion

Bien que je n'aie pas pu finaliser complètement mon projet, il m'a permis d'en apprendre davantage sur le fonctionnement de différentes catégories de la sécurité informatique.

J'aimerais, à terme, le rendre totalement fonctionnel afin de pouvoir l'utiliser au quotidien, et pourquoi pas le proposer à d'autres utilisateurs à la recherche d'un gestionnaire de mots de passe open source.

7. Annexes

7.1 Lien Github

Lien Github du projet : https://github.com/sn1675/Pass

Dedans vous pouvez y trouver un fichier README.md qui explique comment compiler et ou lancer l'application.

7.2 Sources :

Shéma et théorie :

- https://wikipedia.org
- https://www.lucidchart.com

Documentation et solutions :

- https://claude.ai/
- https://openclassrooms.com/fr/
- https://stackoverflow.com/questions
- https://doc.qt.io/